

Practice questions

Instructors: Tanya Goyal

Topics: Feed Forward Neural Networks, Error Backpropagation

Logistics:

- This is an **ungraded** and **optional** practice question set.
- We will make the solutions available in 2-3 days after you have had a chance to attempt them.

Section Feedforward Neural Networks

(1) Consider the following feed forward neural network (FFNN):

$$FFNN(x) = \max(0, (xE)W + b_1)U + b_2 \quad (0.1)$$

Here, x is a one-hot encoding representing the input word. E is the embedding matrix of size $|V| \times d$, where V is the vocabulary size. W is a matrix of size $d \times d_1$ and U is a matrix of size $d_1 \times d$.

(a) What is the size of the bias terms b_1 and b_2 ?

(Solution) If you reason about the matrix multiplications, the dimensions of the input that you are given etc, you'll get the following:

b_1 is of dimension $1 \times d_1$

b_2 is of dimension $1 \times d$

Longer explanation: note that the equation contains xE . We know E has the dimension $|V| \times d$, so x must have dimension $1 \times |V|$. The equation adds $(xE)W$ with b_1 , so these must have the same dimension. It is straightforward to see that $(xE)W$ will have dimension $1 \times d_1$ from the above information and that provided in the question.

(b) For the above FFNN, which of the following is a **parameter**, **hyperparameter**, or **neither**? Write your answer for each of the below items.

1. Vocabulary Size $|V|$ **hyperparameter**
2. Size of the weight matrix W **hyperparameter**
3. Weights of the weight matrix W **parameter**
4. Size of the weight matrix U **hyperparameter**
5. Weights of the weight matrix U **parameter**
6. Bias b_1 of linear layer W **parameter**
7. Input x **neither**
8. Hidden layer output h **neither**

(2) What is **one** important advantage of FFNN language models over n-gram-based language models?

(Solution)

Better representation of context since it offers a generalization over similar contexts.
Can employ a longer history without running into sparse data problems.
They are more accurate.

(4) Suppose you are training an FFNN-based sentiment classification model. The input is a sentence $\mathbf{x} = \langle x_1 x_2 \dots x_n \rangle$ and the output should be a sentiment label $y \in \{+ve, -ve, neutral\}$ (i.e., positive, negative, neutral).

(a) Draw a 2-layer FFNN to handle this sentiment classification task. Additionally, write down the equation detailing the feed forward computation, similar to equation 0.1 above.

Hint: Think about the fact that (i) the input is a sequence of words and not a single word, how should we deal with this? Our network should be able to ingest inputs of different lengths; and (ii) our output should be the label y and not probability distributions.

(Solution) Use a deep averaging network. Refer to class notes.

(b) How can we define the loss to train this model? Remember that your loss should be differentiable.

(Solution) Negative Log Likelihood.

Section FFNNs and Error Backpropagation

1 Computation Graphs for Error Backpropagation. For function $L = b \times (4a + c)$, use the computation graph formalism to compute $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$ and $\frac{\partial L}{\partial c}$.

(Solution)

(2) Computation Graphs. Draw the computation graph for a fully connected FFNN that performs binary classification. It should consist of an input layer of two input nodes, x_1 and x_2 ; no hidden layer; an output layer of one node that employs a sigmoid activation. Use a binary cross entropy loss function.